SEMANA 13 · ARQUITECTURA EN LA NUBE

OpenStack II: De la Teoría al Laboratorio

Parte 1 de 4 · Unidad 3: Nube Privada y Automatización

Docente: **Ing. Rodolfo José Cañas Cervantes** · Universidad de la Costa

De los conceptos arquitectónicos a la práctica experimental con DevStack en una sola máquina virtual.

La Nube Privada como Sistema Coordinado

En la Semana 12 descubrimos que OpenStack no es un monolito sino un conjunto de microservicios especializados.



Horizon Dashboard y Clientes CLI / REST

Punto de entrada unificado para administradores y usuarios inquilinos (Tenants). Solicita autenticación inicial a Keystone.

Autoservicio Web



Keystone (Identidad y Catálogo)

- **Autentica usuarios y emite tokens Fernet**
- **Mantiene el catálogo de endpoints de APIs**
- **Define proyectos, roles y políticas RBAC**



Nova + Placement (Cómputo)

- **Orquesta el ciclo de vida de instancias**
- **Scheduler selecciona el hipervisor óptimo**
- **Gestiona KVM y sabores de hardware (Flavors)**



Neutron (Redes Definidas por Software)

- **Crea redes privadas L2 y subredes CIDR**
- **Enrutadores virtuales L3 con SNAT y Floating IPs**
- **Cortafuegos de capa 4 (Security Groups)**



Glance (Catálogo de Imágenes de SO)

Almacena plantillas de disco QCOW2 y RAW. En el lab usaremos CirrOS, una imagen ultraligera de menos de 20 megabytes para pruebas rápidas.

Consumo mínimo de almacenamiento y transferencia instantánea



Cinder (Almacenamiento Persistente en Bloque)

Gestiona volúmenes persistentes desacoplados de la VM. En laboratorios iniciales las VMs operan sobre disco efímero gestionado directamente por Nova.

Persistencia desacoplada para bases de datos y cargas críticas

Fuente: docs.openstack.org/install-guide/get-started-logical-architecture.html

DESAFÍO DIDÁCTICO

El Reto de Experimentar con OpenStack

Desplegar OpenStack para producción requiere clústeres complejos; en la academia necesitamos agilidad.

Despliegue Tradicional de Producción

Infraestructura Física Extensa
Mínimo 3 controladores HA, nodos cómputo KVM y Ceph.

Redes Físicas Complejas (Underlay)
Switches ToR 10/25 GbE, VLANs troncales, LACP y BGP.

Orquestación Robusta pero Pesada
Kolla-Ansible, OpenStack-Ansible y semanas de despliegue.

Costo y Barrera de Entrada
Inviable en la estación de trabajo de un estudiante.

Alta disponibilidad empresarial a costa de gran complejidad

Enfoque Ágil: Laboratorio All-in-One

Una Única Máquina Virtual de 8 GB RAM
Corre en cualquier PC estándar con VirtualBox o VMware.

Red Virtualizada con Open vSwitch / OVN
Simula switches de acceso, routers y redes públicas.

Instalación Totalmente Automatizada
Un script descarga código oficial, configura BD y lanza.

Mismas APIs y Experiencia Real
Horizon Dashboard y CLI son idénticos a los de producción.

Aprendizaje acelerado sin costos de servidores físicos dedicados

Fuente: docs.openstack.org/devstack/latest/overview.html

HERRAMIENTA OFICIAL

¿Qué es DevStack y Cuál es su Propósito?

La suite de automatización creada por la comunidad OpenInfra para desarrollo, pruebas e instrucción.

Proyecto Oficial DevStack (OpenInfra Foundation)

Repositorio mantenido en opendev.org/openstack/devstack. Scripts Bash de aprovisionamiento declarativo.

1. Pruebas Continuas (CI/CD)

• Motor de pruebas de OpenInfra (Zuul)

Cada parche propuesto en OpenStack se prueba desplegando una nube efímera con DevStack.

• Integración de Tempest

Ejecuta miles de llamadas API automatizadas para garantizar retrocompatibilidad.

2. Desarrollo de Software

• Instalación desde Git

Descarga directamente el código fuente de los repositorios oficiales en /opt/stack.

• Depuración en Tiempo Real

Permite a los desarrolladores modificar el código en Python y reiniciar los servicios al instante.

3. Docencia e Investigación

• Laboratorios Académicos

Estudiantes de arquitectura de nube pueden interactuar con todos los componentes reales.

• Inspección de Configuración

Acceso transparente a las bases de datos SQL, archivos de configuración y colas de mensajes.

DevStack es una herramienta de código abierto gobernada por la OpenInfra Foundation

No es un software empaquetado cerrado: es una colección de guiones de shell interactivos que orquestan el ecosistema.

Garantiza total fidelidad con la especificación y comportamiento de OpenStack upstream

Fuente: docs.openstack.org/devstack/latest/

ADVERTENCIA DE INGENIERÍA

Lo que NO es DevStack: Límites de Diseño

DevStack está diseñado para pruebas y aprendizaje; usarlo en producción viola principios de arquitectura.

ADVERTENCIA OFICIAL DE LA DOCUMENTACIÓN DE OPENSTACK

«DevStack no está pensado para entornos de producción. No intentes operar cargas de trabajo críticas sobre él.»

1. Sin Alta Disponibilidad (Single Point of Failure)

Todos los componentes residen en una sola máquina virtual.

- Si la máquina virtual falla, cae el plano de control y el de datos.
- No existe replicación de MariaDB ni clustering de RabbitMQ.
- No hay tolerancia a fallos ni conmutación por error (Failover).

Riesgo Crítico

2. Volatilidad y Falta de Persistencia

Diseñado intencionalmente como entorno descartable.

- Los servicios no están configurados para auto-inicio tras reboot.
- Al reiniciar Ubuntu, OpenStack queda inoperativo por diseño.
- Requiere reejecutar stack.sh o restaurar snapshots de hipervisor

No Persistente

3. Políticas de Seguridad Relajadas

Prioriza la depuración rápida sobre el endurecimiento defensivo.

- Contraseñas maestras almacenadas en texto plano en local.conf.
- Usuario stack con permisos de sudo sin clave (NOPASSWD: ALL).
- Certificados TLS deshabilitados por defecto en las llamadas API.
- Inadecuado para datos confidenciales o cumplimiento normativo.

Alternativas Recomendadas para Producción

Herramientas oficiales de despliegue industrial.

- **Kolla-Ansible**: Servicios en contenedores con HA y upgrades.
- **OpenStack-Ansible**: Despliegue en contenedores LXC nativos.
- **Canonical Sunbeam (MicroStack)**: Nube privada en Kubernetes.
- Soporte de múltiples zonas de disponibilidad y almacenamiento Ceph.

Fuente: docs.openstack.org/devstack/latest/overview.html

Topología All-in-One: Todo en una Sola VM

Cómo empaquetamos el plano de control, el cómputo y las redes virtuales en un único host virtualizado.

PC Físico del Estudiante (Windows 11 / macOS / Linux) · 16 GB RAM · Hipervisor de Escritorio

Máquina Virtual Invitada: Ubuntu Server 24.04 LTS (All-in-One DevStack) · 8 GB RAM · 4 vCPUs

Plano de Control y Datos

- **MariaDB 10.x (Bases SQL)**
  Esquemas keystone, nova, neutron...
- **RabbitMQ (Broker AMQP)**
  Colas rpc para Nova y agentes
- **Keystone & Horizon**
  Apache2 sirviendo dashboard en puerto 80
- **Nova API & Conductor**
  Gestión centralizada de peticiones
- **Glance API & Registro**
  Imágenes almacenadas en /opt/stack/data

Plano de Cómputo (KVM)

- **Demonio nova-compute**
  Supervisa el estado del hipervisor
- **libvirtd & qemu-kvm**
  Ejecuta las VMs anidadas con aceleración

Instancia CirrOS (VM Anidada)

- Flavor m1.tiny (512 MB RAM, 1 vCPU)
- IP Privada: 192.168.50.x (lab-net)
- IP Flotante: 172.24.4.x (public)
- **Acceso por SSH con lab-key.pem**

Plano de Red (Neutron + OVN)

- **Neutron Server & OVN Controller**
  Traducción de reglas lógicas a flujos OpenFlow
- **Puente de Integración br-int**
  Conecta las interfaces vNIC de las VMs
- **Puente Externo br-ex**
  Contiene la red pública 172.24.4.0/24
- **Enrutador Virtual lab-router**
  Aplica SNAT hacia br-ex y enruta tráfico
- **Security Groups**
  Filtros con conntrack para SSH e ICMP

Fuente: docs.openstack.org/devstack/latest/guides/single-machine.html

Distribución de Procesos y Servicios en la VM

DevStack integra los microservicios de OpenStack bajo unidades de control gestionadas por systemd.

Estructura de Unidades systemd en DevStack: devstack@<servicio>.service

journalctl -u devstack@*

Servicios Base de Infraestructura

mariadb.service
Motor SQL relacional · Puerto TCP 3306

rabbitmq-server.service
Broker de mensajería AMQP · Puerto 5672

apache2.service
Servidor HTTP · Horizon y Keystone WSGI

openvswitch-switch.service
Demonios ovsdb-server y ovs-vswitchd
Base del plano de conmutación de red

Plano de Cómputo e Identidad

devstack@keystone.service
Servicio de identidad y emisión de tokens

devstack@n-api.service
Nova API: recepción de llamadas REST

devstack@n-cpu.service
nova-compute: interacción directa con libvirt

devstack@n-sch.service
nova-scheduler: selección de hipervisores
Coordina con placement-api

Plano de Red e Imágenes

devstack@q-svc.service
neutron-server: API central de redes

devstack@ovn-controller.service
Agente local de OVN en el hipervisor

devstack@g-api.service
Glance API: almacenamiento de imágenes

devstack@placement-api.service
Rastreo cuantitativo de inventario (vCPUs/RAM)
Esencial para la decisión del Scheduler

Fuente: docs.openstack.org/devstack/latest/systemd.html

Dimensionamiento del PC Anfitrión

Capacidad mínima requerida en la máquina física para garantizar estabilidad y rendimiento.

Memoria RAM

16 GB

Mínimo en PC Anfitrión

- **Asignar 8 GB a la VM**
  Requisito duro de DevStack.
- **Reservar 8 GB para el SO**
  Evita que Windows o macOS colapsen.

Con < 8 GB falla pip install

Procesador (CPU)

4 vCPU

Asignados a la VM

- **CPU Host: 4 cores / 8 hilos**
  Intel Core i5/i7 o AMD Ryzen 5/7.
- **VT-x / AMD-V en BIOS**
  Virtualización por hardware obligatoria.

Compilación paralela rápida

Almacenamiento

60 GB

Espacio Libre SSD

- **Disco Virtual: 50 GB**
  Asignación dinámica recomendada.
- **Unidad de Estado Sólido (SSD)**
  Discos HDD mecánicos causan timeouts.

Lectura y escritura intensiva

Conectividad

Internet

Conexión Estable

- **Descarga de 2 a 4 GB**
  Paquetes apt, ruedas Python y Git.
- **Resolución DNS Robusta**
  Uso de servidores 8.8.8.8 o 1.1.1.1.

Evitar cortes durante stack.sh

Impacto de no cumplir los requisitos mínimos:

Si intentan ejecutar DevStack con menos de 8 GB de RAM en la VM, el proceso de instalación fallará inevitablemente cuando el compilador de C o el gestor pip agoten la memoria física y el kernel active el asesino de procesos por falta de memoria (OOM Killer).

Fuente: docs.openstack.org/devstack/latest/guides/single-machine.html

Virtualización Anidada: KVM dentro de la VM

Habilitar la anidación permite que Nova ejecute instancias con aceleración completa por hardware.

Modo Recomendado: Virtualización Anidada Activa

Hipervisor Nivel 1 (VirtualBox / VMware)

Pasa las instrucciones VT-x / AMD-V directamente al procesador virtual.

Ubuntu Server 24.04 (VM)

El kernel detecta /dev/kvm con aceleración directa por hardware.

Nova Compute usa el Driver KVM (virt_type = kvm)

Las instancias CirrOS arrancan en menos de 5 segundos.

Rendimiento nativo y experiencia de nube idéntica a producción.

Modo Fallback: Emulación QEMU por Software

Hipervisor Nivel 1

No expone las extensiones de virtualización a la VM de Ubuntu.

Ubuntu Server 24.04 (VM)

El dispositivo /dev/kvm no existe en el sistema de archivos.

Nova Compute conmuta a emulación (virt_type = qemu)

Cada instrucción de CPU de CirrOS se traduce por software.

Arranque lento (1 a 3 minutos) y alto consumo de CPU host.

Cómo habilitar la virtualización anidada en tu hipervisor:

• **VirtualBox**: Configuración de la VM → Sistema → Procesador → Marcar «Habilitar VT-x/AMD-V anidado».

Si la casilla aparece deshabilitada en gris, ejecutar en terminal: `VBoxManage modifyvm "nombre_vm" --nested-hw-virt on`

• **VMware Workstation**: Ajustes de la VM → Procesadores → Marcar «Virtualizar Intel VT-x/EPT o AMD-V/RVI».

Fuente: docs.openstack.org/nova/latest/admin/support-compute.html

Topologías de Red: Modo Puente vs NAT

Opciones para conectar la máquina virtual anfitriona con tu red física y acceder al dashboard.

Opción A (Recomendada): Adaptador Puente (Bridged)

IP Directa en la Red Local (LAN)

La VM solicita una IP independiente a tu router WiFi (ej. 192.168.1.150).

El PC anfitrión y la VM actúan como dos máquinas vecinas en la red.

Acceso Directo y Transparente

- Horizon se abre directamente en: **`http://192.168.1.150/dashboard`**
- SSH directo a Ubuntu: `ssh cuc@192.168.1.150`
- No requiere configuración de reenvío de puertos (Port Forwarding).

Ventajas Principales

- Cero fricción para acceder a las consolas gráficas web.
- Detección automática de HOST_IP por el script de instalación.
- La experiencia más limpia para entornos residenciales.

Opción B: NAT con Reenvío de Puertos

Aislamiento Tras el Hipervisor

La VM obtiene una IP privada interna (ej. 10.0.2.15) oculta tras el host.

Ideal si la red institucional bloquea múltiples direcciones MAC.

Mapeo de Puertos Obligatorio

- Puerto Host 8080 → Puerto VM 80 (Horizon: localhost:8080/dashboard)
- Puerto Host 2222 → Puerto VM 22 (SSH: ssh -p 2222 cuc@127.0.0.1)
- Requiere configurar las reglas en los ajustes de red del hipervisor.

Particularidad de la Red Flotante

- La red 172.24.4.x vive SOLO dentro de la VM de Ubuntu.
- El SSH a CirrOS se ejecuta desde la propia consola de Ubuntu.
- O mediante un túnel ProxyCommand desde el computador anfitrión.

Fuente: docs.openstack.org/devstack/latest/guides/neutron.html

ESQUEMA DE DIRECCIONAMIENTO

Segmentación IP y Conmutadores Virtuales

Coexistencia de la red de gestión, la red privada de tenants y la red pública simulada.

1. Red de Gestión / LAN Física del Anfitrión

Interfaz eth0 / enp0s3 de la VM Ubuntu · Rango típico: **192.168.1.0/24** (o 10.0.2.0/24 en NAT)

Transporta el tráfico de administración de Horizon (HTTP 80) y la sesión SSH de control del estudiante.

Plano de Gestión

2. Red Pública Simulada de OpenStack (br-ex)

Red física simulada en el puente OVS br-ex · Rango oficial DevStack: **172.24.4.0/24**

- Gateway de la red pública: **172.24.4.1** · Puerta de salida del enrutador lógico hacia el mundo exterior.
- Pool de IPs Flotantes: **172.24.4.2 a 172.24.4.254** asignables a las instancias para acceso entrante.

Red Externa (NAT)

Enrutador L3: lab-router

Aplica Source NAT y mapeo 1:1 de Floating IPs

3. Red Privada Interna de Proyecto: lab-net (br-int)

Subred lógica: **192.168.50.0/24** · Gateway privado: **192.168.50.1** · DNS: 8.8.8.8

Aislamiento de Capa 2 (Geneve / OVN)

Las instancias CirrOS obtienen IP privada mediante DHCP virtual de Neutron.

Instancia CirrOS: mi-primera-vm

IP Fija: 192.168.50.x ↔ Mapeada a IP Flotante: 172.24.4.x

Fuente: docs.openstack.org/neutron/latest/admin/ovn/refarch/refarch.html

RUTA DE TRABAJO

Mapa Metodológico del Laboratorio

Las cuatro fases secuenciales que completaremos para dominar la provisión práctica de infraestructura.

Fase 1: Preparación

Sistema Base
- Instalar Ubuntu Server 24.04
- Configurar red puente o NAT
- Activar virtualización anidada

Usuario del Sistema
- Crear usuario dedicado stack
- Configurar sudoers sin clave

Repositorio Oficial
- Clonar rama stable/2026.1
- Configurar archivo local.conf

Base lista en 15 min

Fase 2: Instalación

Ejecución Automatizada
- Lanzar script stack.sh
- Duración: 30 a 50 minutos
- Supervisar log en tiempo real

Verificación de Éxito
- Banner final con IP y tiempo
- Generación de archivo openrc

Resguardo Preventivo
- Tomar snapshot en hipervisor
- Guardar contraseñas

Evidencia 1: Dashboard

Fase 3: Provisión

Redes y Seguridad
- Crear red y subred lab-net
- Conectar router a public
- Configurar Security Group

Credenciales SSH
- Crear keypair lab-key
- Permisos chmod 600 en .pem

Lanzar Instancia
- Imagen CirrOS + flavor m1.tiny
- Asignar IP flotante pública

Evidencias 2 y 3: VM y SG

Fase 4: Validación

Consola Web noVNC
- Abrir consola en Horizon
- Validar prompt de CirrOS

Conexión SSH Remota
- Conectar con llave lab-key.pem
- Probar comando ip a y ping

Informe y Rúbrica
- Consolidar las 5 evidencias
- Redactar análisis de arquitectura

Evidencias 4 y 5: SSH/Console

Fuente: docs/ESTANDAR-PRODUCCION.md

EVALUACIÓN Y RÚBRICA

Las Cinco Evidencias Demostrativas (10%)

Criterios objetivos para obtener el puntaje completo en el informe práctico de la Semana 13.

Rúbrica Oficial de Calificación · Ponderación: 10 % de la Nota Final de la Asignatura — **100 Puntos Total**

Evidencia 1	**OpenStack Instalado y Operativo en Horizon** Captura de pantalla de la interfaz web Horizon (página Overview o Catálogo de Servicios) con usuario autenticado.	**25 pts**
Evidencia 2	**Instancia CirrOS Activa en Red Propia (lab-net)** Lista de instancias mostrando estado ACTIVE, nombre personalizado con su apellido, IP privada e IP flotante.	**25 pts**
Evidencia 3	**Configuración de Grupo de Seguridad (lab-ssh-icmp)** Detalle de reglas del Security Group demostrando la apertura de ICMP (ping) y TCP puerto 22 (SSH) hacia 0.0.0.0/0.	**15 pts**
Evidencias 4/5	**Acceso por Consola noVNC y por SSH con IP Flotante** Captura de la consola interactiva en Horizon (10 pts) + terminal con sesión SSH ejecutando hostname e ip a (10 pts).	**20 pts**
Reflexión	**Análisis de Equivalencia Conceptual con AWS (Máximo 8 líneas)** Párrafo explicando la correspondencia exacta de cada componente creado (Instancia, VPC, Subnet, IGW, Elastic IP, KeyPair).	**15 pts**

Fuente: actividades/lab-openstack/index.html

CASO ILUSTRATIVO DE REFERENCIA

Validación de Nube Privada en la Industria Local

Escenario aplicado: Cómo el equipo de TI de una terminal portuaria valida OpenStack antes de la inversión física.

Corporación Industrial del Caribe (CIC) · Terminal Portuaria en Barranquilla

Caso ilustrativo de ingeniería: Prototipado ágil con DevStack antes de adquirir infraestructura de cómputo en rack.

Caso Ilustrativo

1. El Problema Operativo

• Operaciones 24/7 en Muelle

Sistemas de pesaje, cámaras OCR y control de contenedores requieren latencia < 5 ms.

• Costos de Nube Pública

El tráfico masivo de telemetría hacia regiones en Estados Unidos incrementa costos de salida de datos.

• Soberanía y Resguardo Local

Cumplimiento de la Ley 1581 de 2012 para almacenamiento de manifiestos de aduana en sitio.

2. Prototipo con DevStack

• Laboratorio de Validación Rápida

El equipo de ingeniería montó DevStack en una estación Dell Precision en la oficina de Barranquilla.

• Pruebas de API y Automatización

Validaron scripts de aprovisionamiento en Python y la integración con los sensores de patio.

• Capacitación Sin Riesgo

Los administradores aprendieron Horizon y Neutron sin poner en peligro los sistemas en producción.

3. Salto a Producción Kolla

• Adquisición Certera de Servidores

Conocían con exactitud la demanda de RAM y vCPUs antes de firmar contratos de hardware.

• Migración Transparente de Scripts

Los mismos comandos openstack probados en DevStack funcionaron en el clúster multinodo.

• Reducción de Riesgo al 100%

Cero improvisación al momento de desplegar la nube privada final con Kolla-Ansible.

Lo que Hemos Construido y Próximos Pasos

Comprendida la arquitectura y los requisitos, en la Parte 2 ejecutaremos la instalación paso a paso.

Lo que Aprendimos en esta Parte 1

El Rol de DevStack en la Nube Privada

Herramienta oficial para desarrollo y docencia; nunca para producción.

Arquitectura All-in-One Autocontenida

Control, cómputo y redes OVN en una sola VM de 8 GB RAM.

Requisitos Técnicos Críticos

16 GB RAM física, virtualización anidada y adaptador en modo puente.

Rúbrica y Metodología Evaluativa

Cinco evidencias claras que aseguran el 10 % de la asignatura.

Lo que Ejecutaremos en la Parte 2

Creación del Usuario del Sistema stack

Políticas de privilegios y permisos en /etc/sudoers.d/50_stack_sh.

Clonación y Fijación de Rama Estable

Uso de stable/2026.1 Gazpacho para garantizar reproducibilidad.

Parametrización Declarativa de local.conf

Variables HOST_IP, contraseñas y activación de Horizon.

Lanzamiento de stack.sh y Solución de Errores

Monitoreo de fases y los primeros cinco errores típicos.

¡Fin del video de la Parte 1! En el siguiente video comenzaremos con la terminal de Ubuntu Server.

SEMANA 13 · ARQUITECTURA EN LA NUBE

Instalación Paso a Paso de DevStack

Parte 2 de 4 · Unidad 3: Automatización y Despliegue All-in-One

Docente: **Ing. Rodolfo José Cañas Cervantes** · Universidad de la Costa

De la consola de Ubuntu Server a una nube OpenStack operativa mediante scripts de automatización.

Preparación de Ubuntu Server 24.04 LTS

Asegurar un sistema operativo limpio, actualizado y sin software conflictivo es el paso cero obligatorio.

Paso 1: Instalación Mínima de Ubuntu 24.04 LTS (Noble Numbat)

Se requiere la versión Server oficial de 64 bits (x86_64). Sin interfaz gráfica de escritorio (GNOME/KDE) para ahorrar RAM.

1. Actualización del Sistema

```
sudo apt update && \
sudo apt dist-upgrade -y
```

Propósito Técnico:

- Parches de seguridad del kernel Linux.
- Actualización de bibliotecas C (glibc).
- Evita incompatibilidades con Python 3.12.

Reiniciar si se actualizó el kernel:

```
sudo reboot
```

2. Paquetes Base Obligatorios

```
sudo apt install -y git \
curl net-tools openssh-server
```

Herramientas Esenciales:

- **git**: Descarga de repositorios OpenStack.
- **curl**: Descarga de CirrOS y llaves GPG.
- **net-tools**: Diagnóstico con ifconfig y route.
- **openssh-server**: Administración remota.

Permite operar vía terminal SSH

3. Limpieza y Verificación

```
ip -4 route get 1.1.1.1
# Detecta IP de salida de la VM
```

Condiciones Previas Críticas:

- No tener Apache ni MariaDB instalados.
- No correr contenedores Docker activos.
- Verificar conectividad con internet:

```
ping -c 3 archive.ubuntu.com
```

Cero servicios conflictivos en el puerto 80

Fuente: docs.openstack.org/devstack/latest/guides/single-machine.html

El Usuario No-Root del Sistema: «stack»

DevStack prohíbe ejecutarse como root; requiere un usuario de servicio con sudo sin contraseña.

REGLA ESTRICTA DE DEVSTACK: ¡JAMÁS EJECUTAR COMO ROOT DIRECTAMENTE!

Privilegios

Comandos de Creación y Configuración

1. Crear usuario dedicado con directorio home:

```
sudo useradd -s /bin/bash -d /opt/stack -m stack
```

2. Otorgar privilegios sudo sin contraseña:

```
echo "stack ALL=(ALL) NOPASSWD: ALL" | \
```

```
sudo tee /etc/sudoers.d/50_stack_sh
```

3. Cambiar a la sesión del usuario stack:

```
sudo su - stack
```

¿Por qué DevStack exige este procedimiento?

Protección del Sistema Operativo

Evita que scripts de compilación de Python alteren permisos críticos del kernel o destruyan la tabla de usuarios raíz.

Instalación No Interactiva

El script stack.sh ejecuta cientos de llamadas sudo.
Si pidiera clave, la instalación se detendría esperando al usuario.

Directorio de Trabajo Estándar (/opt/stack)

El home del usuario stack es exactamente /opt/stack.
Allí se alojarán los repositorios Git, logs y entornos virtuales.

Fuente: docs.openstack.org/devstack/latest/

Clonación y Fijación de Rama: stable/2026.1

Fijar la rama estable Gazpacho garantiza reproducibilidad e impide fallos por cambios en master.

Comando Oficial de Clonación (como usuario stack):

```
git clone https://opendev.org/openstack/devstack -b stable/2026.1 /opt/stack/devstack
```

Rama stable/2026.1 (Gazpacho) · SLURP

Versión Congelada y Probada

Todos los proyectos descargados se alinean automáticamente en la misma rama 2026.1, garantizando compatibilidad mutua.

Cero Roturas de Dependencias

Las versiones de librerías de Python (upper-constraints.txt) están validadas por los tests de integración de OpenInfra.

Reproducibilidad Total

Si ejecutas la instalación hoy o dentro de 3 meses, el resultado será exactamente el mismo en cada computador.

Rama master (Desarrollo Continuo) · Inestable

Código en Desarrollo Diario

Incluye parches experimentales que aún no han sido estabilizados ni certificados para su uso general.

Riesgo Alto de Fallos de Compilación

Un cambio no coordinado en Neutron o Nova puede romper la instalación a la mitad sin explicación aparente.

Inadecuado para Laboratorios

Dos estudiantes que clonen con 2 horas de diferencia podrían obtener comportamientos completamente distintos.

Anatomía de local.conf: Variables Maestras

El archivo único donde se parametriza la red, contraseñas y servicios habilitados de la nube.

/opt/stack/devstack/local.conf

```
[[local|localrc]]

# Direccion IP accesible de la VM
HOST_IP=192.168.1.150

# Contraseñas unificadas para el lab
ADMIN_PASSWORD=CucCloud2026!
DATABASE_PASSWORD=$ADMIN_PASSWORD
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD

# Registro detallado de logs
LOGFILE=/opt/stack/logs/stack.sh.log
LOG_COLOR=False

# Servicios habilitados / deshabilitados
enable_service horizon
disable_service tempest
```

Explicación de Parámetros Clave

[[local|localrc]]

Indica a DevStack que las variables se aplican directamente al archivo localrc.

HOST_IP

La dirección IP de Ubuntu por la que responderán Keystone y Horizon.
Nuestro script la detecta automáticamente con ip route.

ADMIN_PASSWORD y Derivadas

Establece la misma contraseña para el usuario admin de Horizon, la raíz de MariaDB, el broker RabbitMQ y los tokens de comunicación interna de servicios.

enable_service horizon

Fuerza la instalación de la interfaz web gráfica sobre Apache2.

disable_service tempest

Inhabilita la suite de pruebas de integración para ahorrar tiempo y 1.5 GB de RAM.

Permisos del archivo: chmod 600 local.conf (solo lectura para stack)

Fuente: docs.openstack.org/devstack/latest/configuration.html

Servicios Habilitados: Horizon vs Tempest

Optimizar el archivo de configuración reduce a la mitad el tiempo de despliegue y ahorra memoria.

SERVICIO REQUERIDO: enable_service horizon



OpenStack Dashboard (Web UI)

Panel visual de autoservicio para la gestión del laboratorio.

Imprescindible para el Aprendizaje

Permite a los estudiantes visualizar topologías de red, lanzar instancias de forma intuitiva y abrir consolas interactivas.

Despliegue Ligero

Aplicación basada en Django ejecutada bajo Apache mod_wsgi.
Consume apenas 250 MB de memoria RAM.

SERVICIO INNECESARIO: disable_service tempest

CI

OpenStack Integration Test Suite

Batería de pruebas automatizadas para gates de código.

Consumo Excesivo de Recursos

Clona repositorios pesados, descarga árboles de pruebas y consume más de 1.5 GB de memoria RAM en dependencias.

Aumento en el Tiempo de Instalación

Agregar Tempest añade entre 15 y 25 minutos al proceso.
Inhabilitarlo es la mejor decisión para nuestro laboratorio.

Regla de Oro en Arquitectura: Instalar únicamente lo estrictamente necesario

Menos servicios compitiendo por memoria significa mayor estabilidad y menos fallos de OOM en la VM.

Fuente: docs.openstack.org/devstack/latest/configuration.html

Lanzamiento de stack.sh y Supervisión

Iniciando la construcción automatizada y monitoreando las salidas en tiempo real.

```
# Como usuario stack en /opt/stack/devstack:
./stack.sh 2>&1 | tee -a /opt/stack/logs/stack.sh.log
```

1. Comportamiento del Script

• Proceso No Interactivo

No solicita entradas del usuario; toma todas las respuestas directamente de local.conf.

• Duración Estimada

Entre 30 y 50 minutos dependiendo del ancho de banda y la velocidad del disco SSD.

• Sensible a Interrupciones

No cierren la ventana de la terminal ni suspendan la máquina física mientras stack.sh esté corriendo.

2. Supervisión de la Salida

• Patrones en Pantalla

Líneas precedidas de signos más (+) indican funciones Bash en ejecución (set -x).

• Monitoreo en Otra Terminal

```
tail -f /opt/stack/logs/stack.sh.log
```

• Filtrado de Errores

```
grep -i "error" stack.sh.log
```

3. Script Oficial del Curso

• devstack-lab-install.sh

Automatiza la creación de usuarios, clonación, generación de local.conf y lanzamiento.

• Modo Reanudable

Guarda marcas de control para no repetir pasos ya completados con éxito.

• Post-Configuración

Crea la red lab-net, el router, la llave SSH y la imagen CirrOS de forma desatendida.

Fases 1 a 3: Paquetes del SO, Git y Pip

Cómo DevStack resuelve dependencias del sistema operativo y compila el entorno Python.

Fase 1: Verificación de Entorno

Validación de Permisos
Comprueba que el usuario no sea root y tenga sudo.

Detección de Distribución
Verifica que el SO sea Ubuntu 24.04 soportado.

Comprobación de Recursos
Mide la RAM libre y advierte si hay < 8 GB.

Duración: ~2 minutos

Fase 2: Paquetes Apt del Sistema

Instalación de Binarios C/C++
gcc, make, libxml2, libxslt, pkg-config...

Servicios de Soporte
Instala paquetes de MariaDB, RabbitMQ y Apache.

Redes y Virtualización
libvirt-daemon-system, qemu-kvm, openvswitch.

Duración: ~10 minutos

Fase 3: Git y Entornos Pip

Clonación de Proyectos
Descarga keystone, nova, glance, neutron...

Instalación de Ruedas Python
pip install con restricciones upper-constraints.

Compilación de Extensiones
Compila módulos criptográficos (cryptography).

Duración: ~15 minutos

En la Fase 3 es donde ocurre el mayor consumo de memoria y CPU del PC anfitrión

Si el equipo tiene poca memoria, la compilación de librerías Python activará el OOM Killer en este punto.

Fuente: docs.openstack.org/devstack/latest/

Fases 4 y 5: MariaDB, RabbitMQ y Keystone

Puesta en marcha del motor relacional, las colas de mensajería y el servicio de identidad central.

Fases 4 y 5: Inicialización del Estado Persistente y el Bus de Comunicación

1. Base de Datos MariaDB

• Creación de Esquemas SQL

Crea bases keystone, nova, neutron, glance, placement y cinder con cotejamiento utf8mb4.

• Ejecución de db sync

Corre las migraciones de esquemas Alembic para generar cientos de tablas relacionales.

• Cuentas de Usuario

Otorga permisos a cada servicio con la clave definida en DATABASE_PASSWORD.

2. Mensajería RabbitMQ

• Creación de Virtual Host (/)

Configura el espacio de nombres de colas AMQP para la comunicación inter-servicio.

• Usuario de Servicio stack

Crea el usuario AMQP con permisos de lectura y escritura en todas las colas rpc.

• Colas para Nova y Neutron

Permite que nova-api envíe peticiones asíncronas al planificador y a nova-compute.

3. Keystone y Endpoints

• Dominio y Proyecto Default

Crea el dominio Default, el proyecto admin y asigna el rol de administrador al usuario.

• Registro del Catálogo de Servicios

Registra las URLs oficiales de las APIs:

```
http://HOST_IP:5000/v3 (Identity)
http://HOST_IP:8774/v2.1 (Compute)
```

• Claves Fernet

Genera las llaves simétricas para cifrar tokens.

Fuente: docs.openstack.org/keystone/latest/getting-started/architecture.html

Fase 6: Despliegue de Neutron, OVN y Horizon

Construcción de los conmutadores virtuales de red y habilitación del panel web de autoservicio.

Fase 6: Levantamiento del Plano de Datos (OVN) y la Interfaz Gráfica (Horizon)

Plano de Red Definido por Software (OVN)

Creación de Puentes OVS: br-int y br-ex

DevStack crea el switch de integración virtual (br-int) y el puente externo (br-ex) donde residirá la red pública 172.24.4.0/24.

Demonio ovn-controller

Conecta el hipervisor local con las bases de datos Northbound y Southbound de OVN para traducir flujos OpenFlow en microsegundos.

Enrutamiento L3 Distribuido (DVR)

Los enrutadores virtuales se ejecutan directamente en el kernel.

Garantiza conmutación local sin cuellos de botella.

Frontend Gráfico: Horizon Dashboard

Compilación de Activos Estáticos

DevStack ejecuta collectstatic y comprime las plantillas HTML, hojas de estilo CSS y componentes JavaScript de Horizon.

Configuración de Apache2 mod_wsgi

Crea el sitio virtual en /etc/apache2/sites-available/horizon.conf exponiendo la aplicación web en el puerto HTTP estándar 80.

Generación del Archivo de Entorno openrc

Crea /opt/stack/devstack/openrc con las variables OS_*

necesarias para operar OpenStack CLI como administrador.

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

VERIFICACIÓN DE ÉXITO

¿Cómo Saber que la Instalación Terminó Bien?

Las tres señales concluyentes que confirman que OpenStack quedó 100% operativo.

```
# Salida final esperada en la consola de Ubuntu Server:
=========================
DevStack Success: This is your host IP address: 192.168.1.150
Horizon is now available at http://192.168.1.150/dashboard
stack.sh completed in 2415 seconds.
```

1. Banner y Código de Retorno

- **Mensaje «stack.sh completed»**

  Indica que ninguna función Bash retornó un código de error distinto de cero.

- **Código de Salida (echo $?)**

  El comando `echo $?` debe devolver 0.

- **Tiempo Típico**

  Entre 2.000 y 3.000 segundos.

2. Archivo de Entorno openrc

- **Presencia del Archivo**

```
ls -la /opt/stack/devstack/openrc
```

- **Prueba Inmediata con CLI**

```
source openrc admin admin
openstack endpoint list
```

Debe listar todos los endpoints activos

3. Acceso a Horizon (HTTP 80)

- **Prueba con curl Local**

```
curl -I http://localhost/dashboard
```

Debe responder con código HTTP 200 o 302.

- **En el Navegador del PC**

Abrir: `http://<HOST_IP>/dashboard`

Debe renderizar la pantalla de login.

¡Esta captura es la Evidencia 1!

Los Cinco Errores Típicos de DevStack

Diagnóstico y solución para los incidentes más comunes reportados en laboratorio.

Matriz de Diagnóstico Rápido: Causa Raíz y Solución

1. Memoria Insuficiente Proceso «Killed» o fallo pip	**Causa: La VM tiene menos de 8 GB de RAM física asignada o el hipervisor agotó la memoria.** **Solución:** Apagar la VM, subir la RAM a 8192 MB y crear un swapfile de 4 GB: `fallocate -l 4G /swapfile`
2. Corte DNS / Internet git clone o curl fallan	**Causa: Pérdida temporal de conexión wifi o DNS local que no resuelve opendev.org.** **Solución:** Configurar DNS público en /etc/resolv.conf: `nameserver 8.8.8.8` y probar `ping 1.1.1.1`
3. Reejecución sin Limpieza Conflicto en MariaDB/OVS	**Causa: Volver a ejecutar ./stack.sh tras un fallo parcial sin detener los servicios previos.** **Solución:** Ejecutar obligatoriamente `./unstack.sh` antes de relanzar `./stack.sh`
4. Sin Virtualización Anidada Instancia CirrOS en «ERROR»	**Causa: El hipervisor no expone VT-x/AMD-V; libvirt falla al inicializar el dominio KVM.** **Solución:** Activar anidación en VirtualBox/VMware o configurar `virt_type = qemu` en nova-cpu.conf.
5. Reinicio de la VM Horizon no abre tras reboot	**Causa: DevStack no persiste servicios en el arranque por diseño; los demonios quedan inactivos.** **Solución:** Restaurar el snapshot tomado tras stack.sh o volver a correr `./stack.sh` (borrando la marca).

Fuente: actividades/lab-openstack/index.html

Estrategia de Recuperación: unstack vs clean

Herramientas oficiales para reiniciar servicios o limpiar el entorno tras una falla parcial.

./unstack.sh · Detención Controlada

```
cd /opt/stack/devstack && ./unstack.sh
```

Acciones que Realiza:

- Detiene todas las unidades systemd de DevStack.
- Desmonta interfaces virtuales y reglas temporales de iptables.
- Detiene los procesos de Apache2 y mod_wsgi.

Lo que PRESERVA (No destructivo):

- Mantiene las bases de datos MariaDB intactas.
- Mantiene los repositorios Git ya clonados en /opt/stack.
- Mantiene los paquetes y ruedas de Python instalados.

Permite reejecutar ./stack.sh en pocos minutos

./clean.sh · Purga y Limpieza Profunda

```
cd /opt/stack/devstack && ./clean.sh
```

Acciones Destructivas:

- Ejecuta unstack.sh primero de forma automática.
- Elimina todas las bases de datos MariaDB creadas.
- Borra archivos de configuración en /etc/nova, /etc/neutron...

Consecuencia Inmediata:

- Se pierde toda la información de instancias y redes.
- Obliga a reconstruir toda la base relacional desde cero.
- El siguiente ./stack.sh tardará como una instalación nueva.

Usar únicamente como último recurso ante corrupción total

Fuente: docs.openstack.org/devstack/latest/

Credenciales y Snapshot Preventivo Inmediato

Cómo asegurar el éxito de la instalación contra contingencias antes de tocar Horizon.

Paso A: Archivo devstack-credenciales.txt

Generación Automática en el Home del Estudiante:

```
Horizon : http://192.168.1.150/dashboard
Usuario : admin (también existe 'demo')
Clave : CucCloud2026!
Dominio : Default
Proyecto: admin
Llave SSH: $HOME/lab-key.pem (keypair 'lab-key')
```

Seguridad de Acceso:

- Permisos fijados en `chmod 600 devstack-credenciales.txt`.
- Contiene los datos requeridos para ingresar a Horizon en la Parte 3.
- Evita olvidar la contraseña maestra en medio del laboratorio.

Paso B: Snapshot Inmediato del Hipervisor

¡LA ACCIÓN MÁS IMPORTANTE DEL LABORATORIO!

Apenas termine stack.sh, diríjanse al menú de su hipervisor:

- **En VirtualBox:**

  Máquina → Tomar instantánea → Nombre: **«DevStack-Instalado-OK»**

- En VMware: VM → Snapshot → Take Snapshot.

Beneficio Arquitectónico:

- Congela en disco el estado exacto de MariaDB, OVN y Horizon.
- Si la VM se reinicia o se corrompe una red, se restaura en 10 s.
- No tendrán que volver a esperar los 40 minutos de stack.sh.

Póliza de seguro que garantiza el éxito del laboratorio

CIERRE DE PARTE 2

Plataforma Operativa y Salto a Horizon

Con la nube desplegada y respaldada, en la Parte 3 aprovisionaremos nuestra primera VM.

Lo que Logramos en la Parte 2

Ubuntu Server Preparado y Limpio

Usuario stack con sudoers sin contraseña configurado.

Clonación en Rama stable/2026.1

Versión Gazpacho probada y libre de fallos de desarrollo.

Despliegue Exitoso con stack.sh

MariaDB, RabbitMQ, Keystone, Nova, Neutron y Horizon activos.

Snapshot de Seguridad Creado

Plataforma blindada contra apagados o corrupciones de red.

Lo que Viene en la Parte 3

Inicio de Sesión en Horizon Dashboard

Navegación por menús de Proyecto, Cómputo y Redes.

Construcción de Red Privada y Enrutador

Creación de lab-net, subred privada y enlace a la red pública.

Seguridad: Keypair y Security Groups

Apertura de puertos para SSH e ICMP e inyección de llaves.

Lanzamiento de CirrOS y Acceso SSH

Asociación de IP flotante y conexión exitosa por terminal.

¡Fin del video de la Parte 2! En el siguiente video comenzaremos la interacción directa en el navegador.

SEMANA 13 · ARQUITECTURA EN LA NUBE

Horizon Dashboard y tu Primera Máquina Virtual

Parte 3 de 4 · Unidad 3: Autoservicio, Redes SDN e Instancias

Docente: **Ing. Rodolfo José Cañas Cervantes** · Universidad de la Costa

Aprovisionamiento de recursos de infraestructura mediante el panel gráfico y la CLI oficial de OpenStack.

Ingreso a Horizon: http://${HOST_IP}/dashboard

Autenticación web de usuarios contra Keystone a través de la interfaz gráfica oficial.

http://192.168.1.150/dashboard/auth/login/

Credenciales Iniciales

- Dominio: **Default**
- Usuario: **admin**
- Clave: **CucCloud2026!**

(Definida en local.conf)



OpenStack

Panel de Control de la Nube (Horizon)

Dominio

Default

Nombre de usuario

admin

Contraseña

••••••••••••••••

Conectarse

Evidencia 1 (25 Puntos)

Esta pantalla de acceso o la vista del Dashboard con el usuario logueado **constituye la Evidencia 1.**

Fuente: docs.openstack.org/horizon/latest/user/

Aislamiento Multitenant y Vista del Proyecto

Horizon organiza los recursos bajo el concepto de Proyecto (Tenant) con límites estrictos de cuota.

Contexto de Proyecto en Horizon: Proyecto «admin»

Todo recurso (VM, red, volumen, IP flotante) pertenece a un Proyecto específico para garantizar aislamiento multi-inquilino.

Aislamiento Lógico

Cómputo: vCPUs Utilizadas

1 / 20

vCPUs asignadas en el proyecto

Límite por defecto: 20 núcleos virtuales

Memoria RAM Utilizada

512 MB / 50 GB

Consumo de la instancia CirrOS

Margen suficiente para pruebas de lab

IPs Flotantes Asignadas

1 / 50

Direcciones IPv4 públicas reservadas

Pool 172.24.4.0/24 administrado por Neutron

Navegación en el Menú Lateral Izquierdo de Horizon:

- **Proyecto → Cómputo**: Instancias, Imágenes, Sabores (Flavors), Pares de Claves (Key Pairs).
- **Proyecto → Red**: Topología de Red, Redes, Enrutadores (Routers), Grupos de Seguridad, IPs Flotantes.
- **Identidad**: Gestión de Proyectos, Usuarios, Grupos y Roles (visible para administradores).

El menú lateral refleja exactamente la división de microservicios de OpenStack que estudiamos en la teoría.

Fuente: docs.openstack.org/horizon/latest/user/

Creación de Red y Subred: lab-net

Definiendo el switch virtual privado L2 y el rango CIDR con servicio DHCP en Neutron.

Ruta: Proyecto → Red → Redes → Botón «+ Crear Red»

Paso 1: Parámetros de Red (Capa 2)

Nombre de la Red:

lab-net

Estado de Administración:

UP (Marcado)

Crear Subred:

Sí (Marcado)

Neutron crea un conmutador virtual lógico en OVN.
Totalmente aislado de otras redes de inquilinos.

Paso 2: Parámetros de Subred (Capa 3)

Nombre de la Subred:

lab-subnet

Dirección de Red (Formato CIDR):

192.168.50.0/24

Puerta de Enlace (Gateway IP):

192.168.50.1 (Asignada automáticamente)

Servidores de Nombres de Dominio (DNS):

8.8.8.8

Fuente: docs.openstack.org/neutron/latest/admin/intro-os-networking.html

Creación del Enrutador L3: lab-router

Interconectando la subred privada con la red pública para habilitar salida a Internet y Floating IPs.

Ruta: Proyecto → Red → Enrutadores → Botón «+ Crear Enrutador»

1. Crear el Enrutador

Nombre del Enrutador:

lab-router

Estado Administrativo:

UP (Activo)

Red Externa (Gateway):

public (172.24.4.0/24)

Enlace con Gateway Externo:

Conecta el router a br-ex para habilitar SNAT.

2. Agregar Interfaz Interna

Seleccionar Enrutador:

Entrar a lab-router → Pestaña Interfaces.

Pulsar «+ Agregar Interfaz»:

Seleccionar la subred del proyecto:

lab-net: 192.168.50.0/24 (lab-subnet)

IP de la Interfaz:

192.168.50.1 (Por defecto)

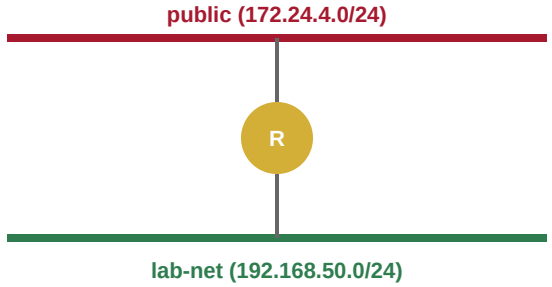
Puente Lógico Completado:

lab-subnet queda conectada al enrutador.

3. Vista en Topología de Red

Menú: Red → Topología de Red

Horizon grafica interactivamente la topología:



Equivalente AWS:

Equivale al Internet Gateway (IGW) y Route Table.

Fuente: docs.openstack.org/neutron/latest/admin/deploy-ovs-selfservice.html

Grupo de Seguridad: Reglas SSH e ICMP

Cortafuegos con estado a nivel de puerto virtual para permitir ping y acceso remoto seguro.

Grupo de Seguridad: lab-ssh-icmp (Evidencia 3: 15 Puntos)

Firewall Virtual

Regla 1: Protocolo ICMP (Diagnóstico con Ping)

- **Dirección**: Entrada (Ingress)
- **Regla / Protocolo**: Todos los ICMP (IPv4)
- **Tipo / Código**: -1 / -1 (Permite Echo Request / Reply)
- **Origen Remoto**: CIDR `0.0.0.0/0`

Permite comprobar conectividad básica mediante ping hacia la IP flotante

Regla 2: Protocolo TCP Puerto 22 (Acceso SSH)

- **Dirección**: Entrada (Ingress)
- **Regla / Protocolo**: TCP Personalizado
- **Rango de Puertos**: Puerto 22 (SSH estándar)
- **Origen Remoto**: CIDR `0.0.0.0/0`

Permite abrir la sesión de terminal segura hacia la instancia CirrOS

Seguridad por Defecto (Zero Trust):

En OpenStack, todo el tráfico entrante está bloqueado por defecto. Si olvidan asociar este grupo de seguridad a la instancia, la máquina virtual encenderá correctamente pero no responderá a pings ni permitirá conexiones SSH («Connection refused» / timeout).

La captura de pantalla de estas dos reglas configuradas en Horizon constituye la Evidencia 3 de su informe técnico.

Fuente: docs.openstack.org/horizon/latest/user/configure-access-and-security-for-instances.html

Par de Claves Criptográficas: lab-key.pem

Inyección de claves públicas vía cloud-init para erradicar el uso de contraseñas vulnerables.

Ruta: Proyecto → Cómputo → Pares de Claves → «+ Crear Par de Claves»

Clave Pública (Almacenada en Nova)

- **Nombre del Keypair:** lab-key
- Tipo de clave: SSH Key (RSA 2048 o ED25519)
- **Inyección Automática con cloud-init:**

Al arrancar, la instancia CirrOS consulta el servicio de metadatos de Nova (169.254.169.254) y deposita la llave pública en:

`/home/cirros/.ssh/authorized_keys`

Clave Privada (Descargada: lab-key.pem)

- **Descarga Única e Irrepetible:**

Horizon entrega el archivo lab-key.pem al crearlo.
OpenStack no almacena copias de la clave privada.

Regla de Permisos Obligatoria en Linux/macOS:

```
chmod 600 ~/lab-key.pem
```

Si los permisos son abiertos, el cliente SSH rechazará la llave por insegura.

Ventaja de Ingeniería frente al uso de contraseñas tradicionales:

La criptografía asimétrica elimina por completo los ataques de fuerza bruta al puerto 22 y permite aprovisionar cientos de servidores en paralelo sin tener que gestionar claves maestras compartidas entre administradores.

Fuente: docs.openstack.org/nova/latest/user/metadata.html

La Imagen CirrOS en Glance: Ultraligera y Rápida

Sistema operativo de pruebas de menos de 20 MB diseñado para validación de nubes IaaS.



Glance Image Service: cirros-0.6.2-x86_64-disk.img

< 20 MB Disco

1. Formato de Disco QCOW2

- **QEMU Copy-On-Write**

Formato optimizado que solo almacena los sectores modificados del disco.

- **Clonación Instantánea**

Nova no necesita duplicar 20 GB de disco; crea una capa delta sobre la imagen base.

- **Compresión Nativa**

La imagen completa pesa apenas 16 MB, ahorrando espacio de almacenamiento.

2. Kernel Minimalista + BusyBox

- **Entorno POSIX Completo**

Incluye herramientas estándar: sh, ifconfig, ip, ping, route, curl, vi y cliente SSH.

- **Huella de RAM Mínima**

Opera con solo 64 MB de memoria RAM (el flavor m1.tiny le asigna 512 MB de sobra).

- **Tiempo de Arranque**

Inicia el kernel y la red en menos de 4 s, ideal para pruebas rápidas de laboratorio.

3. Soporte Nativo de cloud-init

- **Usuario por Defecto:**

`cirros`

- **Clave de Consola por Defecto:**

```
gocubsgo (Solo consola noVNC)
```

- **Inyección de Llave SSH:**

Permite ingresar sin contraseña usando nuestro archivo lab-key.pem descargado.

Fuente: download.cirros-cloud.net/

Selección del Flavor: m1.tiny

Dimensionamiento de recursos de cómputo virtuales: vCPUs, memoria RAM y disco raíz.

Catálogo Estándar de Sabores (Flavors) en Nova Compute

Nombre del Flavor	vCPUs	Memoria RAM	Disco Raíz	Caso de Uso Típico
m1.tiny (Elegido)	**1 vCPU**	**512 MB**	**1 GB**	**Pruebas CirrOS y Validación Lab**
m1.small	1 vCPU	2.048 MB (2 GB)	20 GB	Instancias Linux ligeras (Ubuntu base)
m1.medium	2 vCPUs	4.096 MB (4 GB)	40 GB	Servidores web y aplicaciones medias
m1.large	4 vCPUs	8.192 MB (8 GB)	80 GB	Bases de datos y cargas pesadas

¿Por qué es indispensable usar m1.tiny en nuestro laboratorio All-in-One?

Nuestra VM de Ubuntu cuenta con 8 GB de RAM compartida entre MariaDB, RabbitMQ, Neutron y Nova.
El sabor m1.tiny consume apenas 512 MB de RAM, garantizando que el hipervisor KVM no compita con los servicios de control.

Fuente: docs.openstack.org/nova/latest/user/flavors.html

El Asistente de Horizon: Lanzar Instancia

Integración paso a paso de origen, sabor, red, grupo de seguridad y clave en un solo formulario.

Ruta: Proyecto → Cómputo → Instancias → Botón «Lanzar Instancia»

1. Detalles

Nombre:
vm-tuapellido

Debe incluir tu apellido visible para la rúbrica.

Recuento: 1

2. Fuente

Tipo de Origen:
Imagen

Seleccionar:
cirros

Crear Volumen:
No (Disco efímero)

3. Sabor

Asignar Flavor:
m1.tiny

- 1 vCPU
- 512 MB RAM
- 1 GB Disco

Aprobado por cuota

4. Redes

Red Asignada:
lab-net

- Subred privada 192.168.50.0/24

¡NO marcar la red public directamente!

5. Seguridad

Security Group:
lab-ssh-icmp

Par de Claves:
lab-key

Inyectará llave pública en CirrOS.

Pulsar el botón azul: «Lanzar Instancia»

Nova toma el control, consulta a Placement, descarga la imagen desde Glance y arranca el dominio KVM en segundos.

Fuente: docs.openstack.org/horizon/latest/user/launch-instances.html

Asignación de IP Flotante: Pool 172.24.4.x

Mapeo NAT 1:1 desde la red externa simulada hacia la dirección IP privada de la VM.

Mecanismo de Floating IP: Asociación NAT 1:1 en Neutron OVN

1. Menú Acciones de la Instancia

En la lista de Instancias:

Desplegar el menú derecho de la VM.

«Asociar IP Flotante»

Abre el diálogo modal de reserva
y vinculación de puertos virtuales.

2. Asignar Dirección del Pool

Pool de Red:

public (172.24.4.0/24)

IP Obtenida (Ejemplo):

172.24.4.15

Neutron reserva una IP libre del rango público.

3. Asociar al Puerto Privado

Puerto de la Instancia:

vm-tuapellido: 192.168.50.x

Resultado Inmediato:

Neutron escribe las reglas de NAT en OVN.
Todo paquete a 172.24.4.15 se traduce
al puerto interno 192.168.50.x sin tocar la VM.

Evidencia 2 (25 Puntos): Instancia CirrOS Activa con IP Privada e IP Flotante

La tabla de instancias de Horizon debe mostrar: Nombre con su apellido, Imagen «cirros», IP Privada («192.168.50.x»), IP Flotante («172.24.4.x»), Estado «ACTIVE» y Estado de Potencia «En ejecución». ¡Esta captura completa la Evidencia 2!

Fuente: docs.openstack.org/horizon/latest/user/manage-floating-ip-addresses.html

VERIFICACIÓN OPERATIVA

Conexión a la VM: Consola noVNC y Terminal SSH

Comprobación de acceso por consola gráfica interactiva y sesión de terminal criptográfica.

Evidencia 4 (10 pts): Consola noVNC en Horizon

Ruta: Clic en la instancia → Pestaña «Consola»

```
CirrOS 0.6.2
cirros login: cirros
Password: gocubsgo

$ hostname
vm-tuapellido

$ uptime
14:22:05 up 3 min, 1 user, load average: 0.00
```

Acceso Fuera de Banda (Out-of-Band):

Funciona incluso si la red de la VM falla o el firewall bloquea el puerto 22. Acceso directo al puerto serie virtual.

Evidencia 5 (10 pts): Sesión SSH con IP Flotante

Comando SSH desde la terminal:

```
$ ssh -i ~/lab-key.pem cirros@172.24.4.15

$ hostname
vm-tuapellido

$ ip -4 a show dev eth0
inet 192.168.50.14/24 brd 192.168.50.255

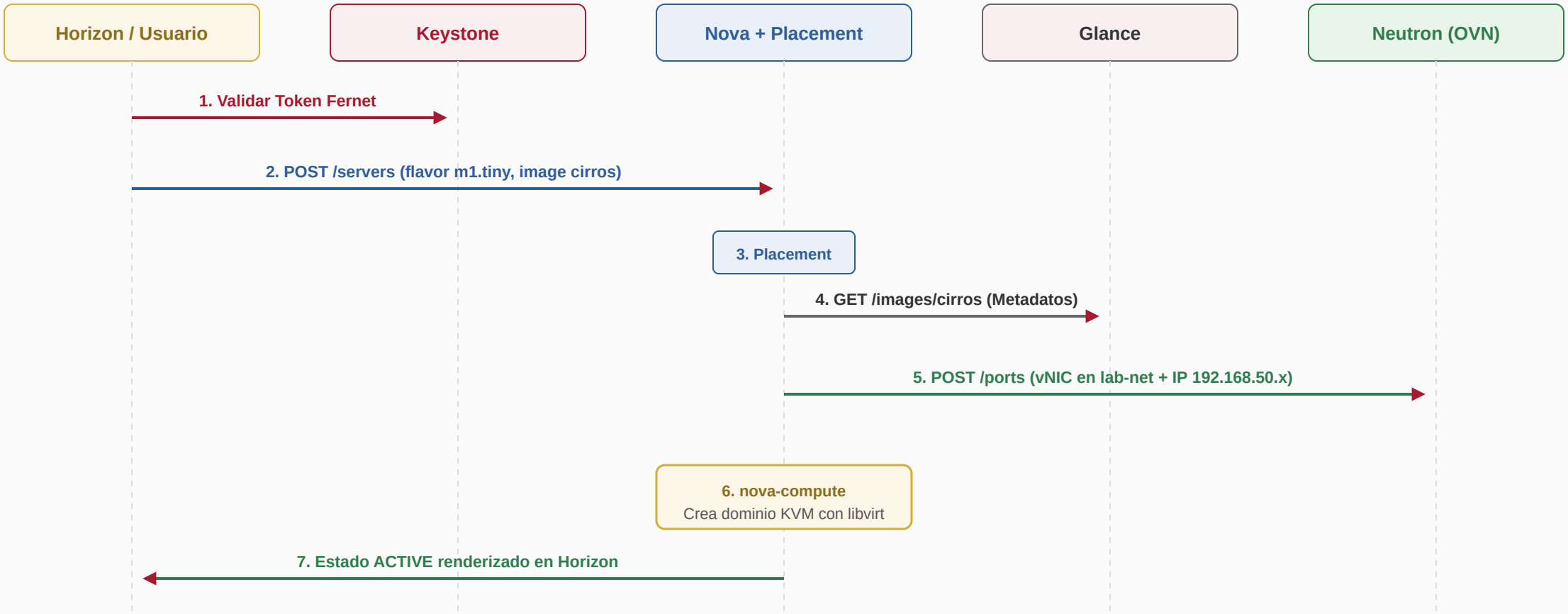
$ ping -c 2 8.8.8.8
2 packets transmitted, 2 received, 0% loss
```

Validación de Extremo a Extremo:

Demuestra que el Security Group, la Floating IP, el enrutador L3 y la llave SSH operan a la perfección.

El Camino de la Petición: ¿Qué Ocurrió Detrás?

Coreografía milimétrica entre Keystone, Nova, Placement, Glance y Neutron durante el lanzamiento.



Fuente: docs.openstack.org/install-guide/launch-instance.html

El Equivalente en OpenStack CLI

Cómo aprovisionar la misma arquitectura mediante ocho órdenes directas desde la terminal.

Secuencia Scriptable: Toda la Práctica en 8 Comandos de Terminal

```
cuc@ubuntu-devstack:~$ bash

# 1. Autenticarse cargando las variables de entorno de Keystone
source /opt/stack/devstack/openrc admin admin

# 2. Crear red privada y subred CIDR con DNS
openstack network create lab-net
openstack subnet create lab-subnet --network lab-net --subnet-range 192.168.50.0/24 --dns-nameserver 8.8.8.8

# 3. Crear router L3, enlazar gateway externo public y conectar la subred
openstack router create lab-router && openstack router set lab-router --external-gateway public
openstack router add subnet lab-router lab-subnet

# 4. Crear grupo de seguridad y abrir reglas ICMP y SSH (TCP 22)
openstack security group create lab-ssh-icmp --description "Reglas SSH e ICMP"
openstack security group rule create lab-ssh-icmp --protocol icmp
openstack security group rule create lab-ssh-icmp --protocol tcp --dst-port 22

# 5. Lanzar instancia CirrOS y asociar IP Flotante de forma scriptable
openstack server create --image cirros --flavor m1.tiny --network lab-net --security-group lab-ssh-icmp --key-name lab-key vm-tuapellido
FIP=$(openstack floating ip create public -f value -c floating_ip_address)
openstack server add floating ip vm-tuapellido $FIP
```


Fuente: docs.openstack.org/python-openstackclient/latest/cli/command-list.html

CIERRE DE PARTE 3

Instancia Operativa y Próximos Pasos

Con la máquina virtual activa y verificada, en la Parte 4 abordaremos operaciones y alta disponibilidad.

Lo que Logramos en la Parte 3

Ingreso Exitoso a Horizon

Validación de la Evidencia 1 en el panel web oficial.

Topología de Red Completa

Red privada lab-net, router L3 y enlace a public.

Cortafuegos y Criptografía

Grupo lab-ssh-icmp (Evidencia 3) y llave lab-key.pem.

VM Activa con Floating IP

Evidencias 2, 4 y 5 completadas (Dashboard, noVNC y SSH).

Lo que Analizaremos en la Parte 4

Operación del Ciclo de Vida

Comandos stop, start, pause, suspend y lectura de logs.

Manejo de Reinicios y Volatilidad

Qué pasa al reiniciar la VM y cómo restaurar con snapshots.

Alta Disponibilidad y Live Migration

Conceptos de HA empresarial, migración en caliente y evacuate.

Consolidación del Informe y Adelanto S14

Checklist de entrega del 10 % y puente hacia IA en la Nube.

¡Fin del video de la Parte 3! En el último video cerraremos la semana con operaciones avanzadas y evaluación.

SEMANA 13 · ARQUITECTURA EN LA NUBE

Operaciones, Alta Disponibilidad y Evaluación

Parte 4 de 4 · Unidad 3: Operación en Producción y Cierre de Semana

Docente: **Ing. Rodolfo José Cañas Cervantes** · Universidad de la Costa

Gestión del ciclo de vida, contingencia de reinicio, alta disponibilidad empresarial y rúbrica final.

Ciclo de Vida de Instancias: Estados y Comandos

Diferencias técnicas entre apagar, pausar, suspender y reiniciar una máquina virtual en Nova.

Máquina de Estados de Cómputo en Nova (Libvirt / KVM)

Stop / Start (Apagado)

Estado: SHUTOFF

- Libvirt apaga el dominio KVM.
- Libera la memoria RAM del host.
- Mantiene el disco y la IP fija.

```
openstack server stop vm
openstack server start vm
```

Equivalente a Stop en EC2

Pause / Unpause (Pausa)

Estado: PAUSED

- Congela los vCPUs en KVM.
- La RAM permanece reservada.
- Reanudación instantánea (< 1 s).

```
openstack server pause vm
openstack server unpause vm
```

Pausa inmediata sin liberar RAM

Suspend / Resume (Hibernar)

Estado: SUSPENDED

- Vuelca la RAM completa a disco.
- Libera la memoria física del host.
- Reanuda estado exacto de memoria.

```
openstack server suspend vm
openstack server resume vm
```

Hibernación con ahorro de RAM

Reboot (Soft vs Hard)

Reinicio Suave vs Forzado

- **Soft**: Envía señal ACPI al SO.
- **Hard**: Reinicio eléctrico en KVM.
- Mantiene IP privada y flotante.

```
openstack server reboot vm
openstack server reboot --hard vm
```

Reinicio sin perder configuración

Fuente: docs.openstack.org/nova/latest/reference/vm-states.html

Diagnóstico y Observabilidad Operativa

Cómo inspeccionar registros de arranque, eventos de cómputo y bitácoras del sistema con journalctl.

Herramientas Oficiales de Auditoría e Inspección en OpenStack

1. Estado y Metadatos de la VM

```
openstack server list --long
```

```
openstack server show vm-tuapellido
```

Información que Revela:

- Hipervisor anfitrión donde corre (OS-EXT-SRV-ATTR:host).
- Nombre del dominio en libvirt (instance-00000001).
- Mapeo de puertos, IPs privadas y flotantes.
- Mensajes de falla si el estado es ERROR.

2. Registro de Consola Serie

```
openstack console log show vm
```

Lectura del Puerto Serie Virtual:

- Muestra los mensajes dmesg del kernel de CirrOS.
- Registra la inicialización de cloud-init.
- Permite depurar cuando SSH no responde.

```
[ 0.000] Linux version 6.x ...
[ 1.250] eth0: link up
[ 2.100] cloud-init: key injected
[ 2.800] lease for 192.168.50.14
[ 3.200] CirrOS ready
```

3. Bitácoras del Plano de Control

```
journalctl -u devstack@n-cpu -f
```

Monitoreo con journalctl:

- **devstack@n-cpu**: Errores de libvirt y KVM.
- **devstack@q-svc**: Bitácoras del servidor Neutron.
- **devstack@keystone**: Trazas de autenticación.

Observabilidad Integral:

Permite rastrear cada solicitud REST desde que entra a Apache hasta el kernel.

Fuente: docs.openstack.org/nova/latest/admin/support-compute.html

El Problema del Reinicio de la VM Anfitriona

Comprendiendo la volatilidad intencional de DevStack y por qué los servicios no arrancan solos.

ESCENARIO COMÚN: «REINICIÉ MI COMPUTADOR Y HOY OPENSTACK NO ABRE»

Al reiniciar Ubuntu Server, el navegador dice «Conexión rechazada» en el puerto 80 y la consola CLI da error de conexión.

1. Volatilidad de systemd

Unidades No Habilitadas

DevStack no ejecuta `systemctl enable` sobre las unidades devstack@*.service.

Diseño Intencional Efímero

Los desarrolladores de OpenStack desean un entorno limpio en cada encendido para no arrastrar estados residuales corruptos.

El SO sube, pero OpenStack permanece apagado

2. Redes OVS Desincronizadas

Pérdida de Puertos Virtuales

Los puentes br-int y br-ex pierden sus flujos dinámicos y las tablas de enrutamiento OVN.

Cambio de IP en la VM

Si el router asigna una nueva IP por DHCP, la variable HOST_IP de local.conf queda desalineada con el catálogo de Keystone.

Las APIs no pueden resolver los endpoints

3. La Solución Definitiva

Snapshot del Hipervisor

Si tomaste el snapshot preventivo en la Parte 2, basta con pulsar «Restaurar».

Re-ejecución de stack.sh

Si no hay snapshot, se debe eliminar la marca y correr ./stack.sh como usuario stack.

En la siguiente diapositiva:

Veremos el procedimiento exacto de rescate.

Protocolo de Rescate: Snapshot vs Re-Stack

Procedimiento paso a paso para revivir los servicios si la máquina virtual sufrió un reinicio imprevisto.

Dos Caminos para Revivir OpenStack tras un Apagado o Fallo Físico

Camino A (Recomendado): Restaurar Snapshot

Paso 1: Apagar la VM en el Hipervisor

Cerrar la máquina virtual (Power Off) en VirtualBox o VMware.

Paso 2: Revertir al Snapshot

Seleccionar la instantánea «DevStack-Instalado-OK» y pulsar el botón «Restaurar / Revert».

Resultado Inmediato:

La máquina enciende con todos los servicios en memoria activos.
Tiempo de rescate: ¡Solo 15 segundos!

Camino B: Reejecución Controlada de stack.sh

1. Eliminar la marca de control si existe:

```
sudo rm -f /opt/stack/.stack-done-*
```

2. Iniciar sesión como stack y relanzar:

```
sudo su - stack
cd devstack && ./stack.sh
```

Comportamiento:

- No descarga los repositorios Git de nuevo.
- Reconfigura puentes OVS y relanza unidades systemd.

Tarda entre 3 y 8 minutos en estabilizarse.

De All-in-One a Producción: Alta Disponibilidad

Cómo escala OpenStack en centros de datos reales eliminando puntos únicos de falla (HA).

Arquitectura de Alta Disponibilidad Empresarial (Zero SPOF)

1. Balanceo de Carga (VIP)

- **HAProxy + Keepalived**
  Dos o más balanceadores comparten una IP Virtual (VIP) mediante el protocolo VRRP.
- **Distribución Activo-Activo**
  Distribuyen las llamadas REST entrantes entre múltiples instancias de nova-api y keystone.
- **Detección de Caída (Health Checks)**
  Si un nodo controlador muere, el tráfico se redirige al instante sin corte para el usuario.

2. Estado y Mensajería Clustered

- **Clúster MariaDB Galera (3 Nodos)**
  Replicación sincrónica multimaestro basada en certificación de transacciones (quórum impar).
- **Clúster RabbitMQ en Espejo**
  Colas Quorum altamente disponibles distribuidas en los tres controladores maestros.
- **Tolerancia a Particiones de Red**
  Previene el temido escenario de Split-Brain mediante votos mayoritarios de membresía.

3. Cómputo Distribuido y Ceph

- **Decenas de Nodos nova-compute**
  Hipervisores KVM puros sin servicios de control, agrupados en Availability Zones y Host Aggregates.
- **Almacenamiento Ceph RBD**
  Almacenamiento unificado distribuido con triple copia de datos para volúmenes Cinder e imágenes.
- **Habilita Migración en Caliente**
  Permite mover VMs entre servidores físicos sin tiempo de parada (Zero-Downtime).

Fuente: docs.openstack.org/ha-guide/

Migración en Vivo: Live Migration sin Parada

Traslado transparente de instancias entre hipervisores físicos para mantenimiento y balanceo.

Mantenimiento de Infraestructura Transparente: openstack server live-migration

Nodo de Cómputo Origen (compute-01)

Instancia en Ejecución (VM Activa)

Memoria RAM activa atendiendo transacciones web.

Estado: Pre-copia iterativa de páginas sucias (dirty pages).

Motivo del Traslado:

Actualización de kernel Linux o reemplazo de memoria RAM.

Red 10/25 GbE

Copia de RAM en caliente

Nodo de Cómputo Destino (compute-02)

Recepción de Páginas de Memoria

QEMU/KVM replica el mapa de memoria en destino.

Tiempo de corte imperceptible: < 100 milisegundos.

Conmutación de Red Instantánea:

Neutron OVN redirige el tráfico hacia el nuevo puerto.

Comando Oficial de Migración en Vivo (Nova Admin):

```
openstack server live-migration --host compute-02 vm-tuapellido
```


Fuente: docs.openstack.org/nova/latest/admin/live-migration-usage.html

Variantes de Migración: Compartida vs Block

Diferencias arquitectónicas entre migrar sobre Ceph/NFS y migrar con almacenamiento local en bloque.

1. Migración con Almacenamiento Compartido (Ceph / SAN)

Los Discos No se Mueven de Lugar

Tanto el nodo origen como el destino están conectados al mismo pool distribuido de Ceph RBD o almacenamiento NFS compartido.

Solo se Transfiere la Memoria RAM:

- Tráfico de red mínimo (solo páginas de memoria activas).
- Proceso completado en 10 a 30 segundos.

El estándar de las nubes empresariales.

```
openstack server live-migration compute-02 vm
```

2. Block Live Migration (Sin Almacenamiento Compartido)

Discos Locales en Cada Hipervisor (DAS)

Los hipervisores no comparten un backend de almacenamiento común; cada máquina utiliza discos SSD conectados internamente en el nodo.

Se Transfiere Disco Efímero + Memoria RAM:

- QEMU debe copiar los gigabytes del disco completo por la red.
- Alto consumo de ancho de banda y mayor tiempo de traslado.

Requiere el parámetro --block-migrate.

```
openstack server live-migration --block-migrate vm
```


Fuente: docs.openstack.org/nova/latest/admin/live-migration-usage.html

Evacuación de Hosts: Recuperación ante Fallos

Mecanismo para reconstruir instancias automáticamente cuando un hipervisor físico muere por completo.

Recuperación Ante Desastre Físico: openstack server evacuate

1. Falla Catastrófica del Host

Caída del Servidor Físico

- Pérdida total de energía o fallo en placa base.
- El hipervisor compute-01 queda inalcanzable.
- ¡NO es posible hacer Live Migration!

Imposible copiar RAM:
El procesador origen está apagado.

2. Reconstrucción en Destino

Orden de Evacuación

Nova Scheduler selecciona un hipervisor vivo.

Reconstruye la VM a partir del volumen en Ceph.

```
openstack server evacuate vm
```

Reinicia la VM preservando datos e IPs

3. Automatización con Masakari

Servicio de HA de Instancias

- Monitorea la salud de cada hipervisor.
- Dispara la evacuación de forma 100 % autónoma.
- Reduce el tiempo de recuperación (RTO < 2 min).

Resiliencia Automática:
Sin intervención humana en la madrugada.

Diferencia Clave: Live Migration vs Evacuación

- **Live Migration**: Mantenimiento planificado; el nodo origen está sano; cero segundos de caída (Zero Downtime).
- **Evacuación**: Falla de hardware inesperada; el nodo origen murió; implica un reinicio de la VM pero rescata datos e IPs.

Fuente: docs.openstack.org/nova/latest/admin/evacuate.html

ENTREGA DE LABORATORIO

Evaluación del Laboratorio (10% de la Asignatura)

Guía oficial publicada y acceso al material complementario para la entrega técnica.

Actividad Práctica Semana 13 · Guía Oficial Disponible en el Portal del Curso

Especificaciones del Informe Técnico

Formato de Entrega:

Documento PDF individual generado desde procesador de texto.

Estructura del Documento:

1. Portada institucional con nombres completos y código.
2. Resumen de especificaciones de la máquina física y de la VM.
3. Las 5 capturas demostrativas con nombre visible.
4. Párrafo reflexivo de equivalencia conceptual con AWS.

Ponderación Oficial:

10 % de la calificación definitiva del curso (100 puntos netos).
Calificación objetiva contra la rúbrica presentada.

Recursos y Acceso a la Guía

Enlace Directo en el Repositorio:

```
laboratorios/lab-openstack/index.html
```

Script de Automatización Descargable:

```
devstack-lab-install.sh
```

Soporte y Asesoría Técnica:

Canal de dudas en el aula virtual y sesiones de tutoría en los laboratorios de informática de la Universidad de la Costa.

LISTA DE VERIFICACIÓN

Checklist Final Antes de la Entrega

Comprueba cada uno de los puntos obligatorios para asegurar la máxima calificación.

Lista de Chequeo Obligatoria (Checklist de Calidad)

- [ ] **Evidencia 1: Acceso a Horizon Dashboard (25 Puntos)**
  Captura mostrando la barra de navegación con el usuario admin autenticado en http://HOST_IP/dashboard.

- [ ] **Evidencia 2: Tabla de Instancias con Nombre Personalizado (25 Puntos)**
  Instancia CirrOS mostrando estado ACTIVE, IP privada 192.168.50.x e IP flotante 172.24.4.x, con tu apellido visible.

- [ ] **Evidencia 3: Reglas de Cortafuegos en Grupo de Seguridad (15 Puntos)**
  Detalle del grupo lab-ssh-icmp demostrando apertura de ICMP (ping) y TCP puerto 22 (SSH) hacia 0.0.0.0/0.

- [ ] **Evidencias 4 y 5: Acceso por Consola noVNC y por SSH (20 Puntos)**
  Consola interactiva web abierta en CirrOS (10 pts) + terminal con SSH exitoso mostrando hostname e ip a (10 pts).

- [ ] **Análisis Reflexivo: Mapeo Conceptual OpenStack ↔ AWS (15 Puntos)**
  Párrafo de máximo 8 líneas explicando la correspondencia de Instancia, VPC, Subnet, IGW, Elastic IP y KeyPair.

Lecciones de la Corporación Industrial del Caribe

Cómo el laboratorio local ahorró meses de improvisación en el despliegue del puerto de Barranquilla.

Balance de Ingeniería: Del Prototipo DevStack a la Nube de Producción en CIC

Tres lecciones operativas que transformaron el laboratorio experimental en una ventaja competitiva real.

1. Automatización Temprana

- **Prohibido el Clic Manual**

El equipo de CIC utilizó Horizon para entender los conceptos, pero tradujo cada acción a scripts de OpenStack CLI y plantillas Terraform.

- **Despliegues Repetibles**

Al pasar al rack físico, el aprovisionamiento de redes y routers se ejecutó en 45 segundos.

2. Dimensionamiento Real

- **Descubrimiento de Sobrecostos**

Al simular las cargas en DevStack descubrieron que la telemetría requería más IOPs que CPU.

- **Inversión Certera**

Invirtieron presupuesto en discos NVMe para Ceph en lugar de comprar procesadores sobredimensionados.

3. Resiliencia Operativa

- **Cultura de Recuperación**

El manejo de contingencias y snapshots en el lab preparó a los ingenieros para enfrentar caídas.

- **Cero Tiempo de Inactividad**

En producción implementaron Live Migration y Masakari, logrando un 99.95 % de disponibilidad.

Fuente: docs/investigacion-S12-openstack.md

SÍNTESIS CONCEPTUAL

La Semana 13 en Cuatro Ideas Maestras

Los cuatro pilares técnicos fundamentales que consolidamos en esta sesión de laboratorio.

1. DevStack es un Vehículo de Aprendizaje Efímero

All-in-One para Experimentar, Nunca para Producción

Permite correr una nube completa en una sola máquina virtual de 8 GB, compartiendo las mismas APIs REST y CLI que los clústeres gigantescos.

Requiere snapshots inmediatos para protegerse contra reinicios no persistentes.

2. Automatización Declarativa con local.conf y stack.sh

Infraestructura Reproducible desde Código Fuente

Fijar la rama stable/2026.1 Gazpacho garantiza despliegues idénticos.
El script stack.sh resuelve dependencias apt, pip y configura bases MariaDB.

Optimizar servicios (activar Horizon y apagar Tempest) acelera el laboratorio.

3. Redes SDN y Seguridad Zero Trust en Neutron

Aislamiento L2, Enrutamiento L3 y Mapeo NAT 1:1

Las redes privadas (lab-net) se conectan al exterior mediante routers virtuales.
Los Security Groups filtran tráfico a nivel de kernel y las Floating IPs

permiten el acceso entrante por SSH de manera segura con lab-key.pem.

4. Visión Hacia la Alta Disponibilidad Empresarial

Cero Puntos Únicos de Falla y Movilidad de Cargas

En producción escalamos con clústeres MariaDB Galera y almacenamiento Ceph.
La migración en vivo (Live Migration) permite mantenimiento sin caída de servicio

y la evacuación rescata instancias críticas ante desastres de hardware físico.

PUENTE CONCEPTUAL

Adelanto de la Semana 14: IA en la Nube I

De la infraestructura de máquinas virtuales al cómputo acelerado para Inteligencia Artificial.

Próxima Sesión · Semana 14: Inteligencia Artificial en la Nube I

1. Modelos Fundacionales

• De CPUs a GPUs y TPUs

Cómo la nube privada y pública adapta sus hipervisores para cargas de cómputo tensorial.

• Grandes Modelos de Lenguaje

Arquitecturas Transformer, pesos y parámetros como activos de infraestructura.

Cambio de Paradigma:
La IA deja de ser software para ser un servicio base.

2. Tokens y Ventana de Contexto

• La Unidad Métrica de la IA

Comprensión de tokens de entrada vs salida, tarificación por millón y latencia de primer token.

• Gestión de Contexto

Ventanas de 128k a 1M de tokens y su impacto en la memoria VRAM y caché KV de inferencia.

Economía de Nube:
Optimización de costos en llamadas API de IA.

3. Entrenamiento vs Inferencia

• Dos Cargas Totalmente Distintas

Entrenamiento: Semanas de cómputo en clúster.
Inferencia: Milisegundos para servir usuarios.

• Arquitectura de Servicio (AIaaS)

Consumo de modelos vía endpoints REST, desacoplamiento de backend y modelos locales.

Nube Híbrida de IA:
Privacidad local frente a potencia de nube pública.

Agradecimientos y Créditos Institucionales

Universidad de la Costa · Departamento de Computación y TI · Programa de Ingeniería de Sistemas.

UNIVERSIDAD DE LA COSTA · CUC

Arquitectura en la Nube

Semana 13: OpenStack II — Instalación con DevStack y Primera VM

Docente Titular: Ing. Rodolfo José Cañas Cervantes

Departamento: Computación y Tecnologías de la Información

Programa: Ingeniería de Sistemas

Periodo Académico: 2026-2 · Barranquilla, Atlántico, Colombia

¡Felicitaciones por culminar con éxito la Semana 13!

Consolidar su informe con las 5 evidencias les asegura el 10 % de la asignatura.